

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»



Направление подготовки
44.03.04 Профессиональное обучение (по отраслям)

направленность (профиль)
«Информатика и искусственный интеллект в образовании»

Курсовая работа
по дисциплине «Прикладные информационные технологии»

«Развёртывание микросервисной архитектуры с использованием Docker
Compose и защищённого доступа через Let's Encrypt для приложения
AnonChat»

Обучающегося 3 курса
очной формы обучения
Самбуев Алдар Баирович

Руководитель курсовой работы:
старший преподаватель кафедры ИТЭО
Аксютин Павел Александрович

Санкт-Петербург
2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
ГЛАВА 1. ОБЗОР И СРАВНЕНИЕ ТЕХНОЛОГИЙ	4
1.1. Контейнеризация: Docker	4
1.2. Оркестрация: Docker Compose.....	4
1.3. Backend: FastAPI.....	5
1.4. Frontend: React	6
1.5. База данных: PostgreSQL.....	6
1.6. Веб-сервер и обратный прокси: NGINX.....	7
1.7. SSL-сертификаты: Let's Encrypt + Certbot.....	8
ГЛАВА 2. АРХИТЕКТУРА И РЕАЛИЗАЦИЯ	9
2.1. Архитектура приложения.....	9
2.2. Серверная часть (Backend).....	10
2.3. Клиентская часть (Frontend).....	11
2.4. Конфигурация NGINX.....	11
2.5. Выпуск SSL-сертификата.....	12
ГЛАВА 3. ДОКУМЕНТАЦИЯ ПО ЗАПУСКУ	13
Шаг 1. Установить Docker и плагин Compose	13
Шаг 2. Клонировать репозиторий	13
Шаг 4. Выпустить SSL-сертификат.....	13
Шаг 5. Подключить сертификат к NGINX	14
ЗАКЛЮЧЕНИЕ	15
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	16

ВВЕДЕНИЕ

Современные веб-приложения всё чаще строятся по принципу контейнеризации: каждый компонент системы (база данных, серверная логика, клиентская часть, обратный прокси-сервер) упаковывается в изолированный контейнер и управляется независимо. Такой подход упрощает развёртывание, масштабирование и обновление приложений на любой инфраструктуре.

Целью данной курсовой работы является разработка и контейнеризация многокомпонентного веб-приложения AnonChat — анонимного чата в реальном времени — с использованием технологий Docker, Docker Compose, NGINX и автоматического выпуска SSL-сертификатов через Let's Encrypt.

Задачи работы:

- разработать клиентскую часть приложения на React с поддержкой WebSocket;
- разработать серверную часть на FastAPI с REST API и WebSocket-эндпоинтами;
- настроить базу данных PostgreSQL для хранения комнат и истории сообщений;
- написать Dockerfile для каждого сервиса и объединить их в Docker Compose;
- настроить NGINX как обратный прокси-сервер с поддержкой HTTPS;
- автоматизировать выпуск и обновление SSL-сертификатов через Let's Encrypt и Certbot;
- составить документацию по запуску проекта.

ГЛАВА 1. ОБЗОР И СРАВНЕНИЕ ТЕХНОЛОГИЙ

1.1. Контейнеризация: Docker

Docker — платформа для упаковки, распространения и запуска приложений в изолированных контейнерах. В отличие от виртуальных машин, контейнеры используют ядро хост-системы и не требуют запуска полноценной ОС, что делает их значительно легче и быстрее.

Характеристика	Docker (контейнеры)	Виртуальные машины
Время запуска	Секунды	Минуты
Потребление ресурсов	Низкое (общее ядро ОС)	Высокое (своя ОС)
Изоляция	Уровень процессов	Полная аппаратная
Переносимость	Высокая (Docker Hub)	Средняя
Типичное применение	Микросервисы, CI/CD	Разные ОС, legacy ПО

Для данного проекта выбран Docker, поскольку он обеспечивает воспроизводимое окружение и позволяет запустить весь стек одной командой на любом сервере.

1.2. Оркестрация: Docker Compose

Docker Compose — инструмент для описания и запуска многоконтейнерных приложений через единый файл `docker-compose.yml`. Альтернативами являются Kubernetes и Docker Swarm.

Инструмент	Сложность настройки	Масштабируемость	Применение
Docker Compose	Низкая	Один сервер	Разработка, небольшие проекты

Инструмент	Сложность настройки	Масштабируемость	Применение
Docker Swarm	Средняя	Несколько серверов	Простые продакшн-кластеры
Kubernetes	Высокая	Тысячи узлов	Крупные production-системы

Docker Compose выбран как наиболее подходящий инструмент для учебного проекта: минимальная конфигурация, встроенная поддержка переменных окружения и зависимостей между сервисами.

1.3. Backend: FastAPI

FastAPI — современный высокопроизводительный фреймворк для Python, основанный на стандарте ASGI. Обеспечивает нативную поддержку WebSocket, автоматическую генерацию документации OpenAPI и валидацию данных через Pydantic.

Фреймворк	Язык	WebSocket	Производительность	Документация
FastAPI	Python	Нативная	Высокая (async)	Авто (OpenAPI)
Django + Channels	Python	Через Channels	Средняя	Ручная
Express.js	JavaScript	Через ws/socket.io	Высокая	Ручная
NestJS	TypeScript	Через Gateway	Высокая	Декораторы

FastAPI выбран за сочетание простоты кода, нативной поддержки WebSocket и высокой производительности на основе asyncio.

1.4. Frontend: React

React — библиотека для построения пользовательских интерфейсов, разработанная Meta. Компонентная архитектура и хуки позволяют управлять состоянием WebSocket-соединения декларативно, что упрощает разработку чата в реальном времени.

Технология	Подход	Экосистема	Порог входа
React + Vite	Компоненты + хуки	Огромная	Средний
Vue 3	Компоненты + Composition API	Большая	Низкий
Svelte	Компиляция в JS	Средняя	Низкий
Vanilla JS	Нет фреймворка	—	Низкий, но код сложнее

React выбран как наиболее распространённый инструмент с богатой экосистемой. Vite используется вместо Create React App за скорость сборки и простоту конфигурации.

1.5. База данных: PostgreSQL

PostgreSQL — мощная объектно-реляционная СУБД с открытым исходным кодом. Используется для хранения информации о комнатах и истории сообщений.

СУБД	Тип	Надёжность	Сложность	Применение
PostgreSQL	Реляционная	Высокая (ACID)	Средняя	Продакшн, сложные запросы
MySQL / MariaDB	Реляционная	Высокая	Низкая	Веб-приложения

СУБД	Тип	Надёжность	Сложность	Применение
SQLite	Встроенная	Средняя	Очень низкая	Разработка, мобильные
MongoDB	Документная (NoSQL)	Высокая	Средняя	Гибкая схема данных
Redis	In-memory	Средняя	Низкая	Кэш, очереди, pub/sub

PostgreSQL выбран за надёжность, поддержку ACID-транзакций и хорошую интеграцию с SQLAlchemy. Redis рассматривался как альтернатива для хранения сообщений (pub/sub), однако PostgreSQL предпочтителен для постоянного хранения истории чата.

1.6. Веб-сервер и обратный прокси: NGINX

NGINX используется в проекте как обратный прокси-сервер: принимает входящие запросы и перенаправляет их к нужному контейнеру — фронтенду или бэкенду. Особую роль играет корректная проксировка WebSocket-соединений через заголовки Upgrade и Connection.

Решение	WebSocket	SSL Termination	Конфигурация	Нагрузка
NGINX	Ручная настройка	Да	Файловая (conf)	Очень высокая
Traefik	Автоматическая	Авто (Let's Encrypt)	Docker labels	Высокая
Caddy	Автоматическая	Авто (Let's Encrypt)	Caddyfile	Высокая
HAProxy	Да	Да	Файловая	Очень высокая

NGINX выбран как наиболее распространённое и хорошо документированное решение. Traefik и Caddy могут автоматически управлять

сертификатами через Docker-метки, однако NGINX даёт более полный контроль над конфигурацией и является стандартом в индустрии.

1.7. SSL-сертификаты: Let's Encrypt + Certbot

Let's Encrypt — бесплатный центр сертификации, выдающий доверенные SSL/TLS-сертификаты. Certbot — официальный ACME-клиент для автоматического выпуска и обновления сертификатов.

Решение	Стоимость	Автообновление	Доверие браузеров	Wildcard
Let's Encrypt + Certbot	Бесплатно	Да (контейнер)	Да	Да
Коммерческий CA	От \$70/год	Ручное	Да	Да
Самоподписанный	Бесплатно	Вручную	Нет	Да
ZeroSSL	Бесплатно (до 3)	Да (ACME)	Да	Нет

Let's Encrypt выбран как стандарт для автоматического управления сертификатами в контейнеризованных приложениях: бесплатно, доверяется всеми браузерами, обновляется автоматически.

ГЛАВА 2. АРХИТЕКТУРА И РЕАЛИЗАЦИЯ

2.1. Архитектура приложения

Приложение AnonChat состоит из пяти Docker-контейнеров, взаимодействующих через внутреннюю сеть Docker (bridge-сеть app-net). Единственная точка входа извне — контейнер NGINX, открывающий порты 80 и 443.

Контейнер	Образ / Технология	Назначение	Порт
db	postgres:16-alpine	Хранение комнат и сообщений	5432
backend	Python 3.12 + FastAPI	REST API и WebSocket-сервер	8000
frontend	Node 20 + Nginx	React SPA (статика)	80
nginx	nginx:alpine	Обратный прокси, SSL	80, 443
certbot	certbot/certbot	Выпуск и обновление сертификатов	—

Схема взаимодействия компонентов: браузер подключается к NGINX по порту 443 (HTTPS). NGINX распределяет запросы: корневой путь / направляется к контейнеру frontend, путь /api/ — к backend, путь /ws/ — также к backend, но через протокол WebSocket.

Особенностью архитектуры является поддержка двух протоколов через один NGINX: обычный HTTP/HTTPS для REST API и фронтенда, а также постоянное WebSocket-соединение для обмена сообщениями в реальном времени. Для WebSocket в конфигурации NGINX необходимо явно передавать заголовки Upgrade и Connection, а также устанавливать увеличенный timeout (proxy_read_timeout 86400).

2.2. Серверная часть (Backend)

Серверная часть реализована на FastAPI. Приложение предоставляет три группы эндпоинтов: REST API для управления комнатами, REST API для получения истории сообщений и WebSocket-эндпоинт для обмена сообщениями в реальном времени.

Для управления множеством одновременных WebSocket-соединений реализован класс `ConnectionManager`, который хранит список активных подключений для каждой комнаты и рассылает сообщения всем участникам через метод `broadcast`.

Для работы с базой данных используется SQLAlchemy ORM с двумя моделями:

- Room — комната чата (код, название, дата создания);
- Message — сообщение (код комнаты, никнейм автора, текст, временная метка).

Метод	URL	Описание
POST	/api/rooms	Создать новую комнату, вернуть уникальный 6-значный код
GET	/api/rooms/{code}	Получить информацию о комнате по коду
GET	/api/rooms/{code}/messages	История последних 50 сообщений
GET	/api/health	Проверка работоспособности сервиса
WS	/ws/{room_code}/{username}	WebSocket-соединение для чата

2.3. Клиентская часть (Frontend)

Клиентская часть реализована на React 18 с использованием хуков `useState`, `useEffect` и `useRef`. Приложение состоит из двух экранов:

- экран входа (`JoinScreen`) — форма с вкладками для создания новой комнаты или входа по коду;
- экран чата (`ChatScreen`) — список сообщений, поле ввода, заголовок с кодом комнаты.

WebSocket-соединение устанавливается в `useEffect` при монтировании компонента `ChatScreen`. При получении сообщения от сервера состояние `messages` обновляется, что вызывает перерисовку списка. Автоскролл к последнему сообщению реализован через `useRef` на вспомогательный элемент в конце списка.

Сборка фронтенда выполняется в многоступенчатом (multi-stage) `Dockerfile`: первая стадия устанавливает зависимости и собирает статику через `Vite`, вторая стадия копирует готовую сборку в образ `nginx:alpine`. Это позволяет уменьшить итоговый размер образа в несколько раз.

2.4. Конфигурация NGINX

NGINX выполняет роль обратного прокси и завершает SSL. Конфигурация включает два `server`-блока:

- HTTP (порт 80): перенаправляет все запросы на HTTPS, кроме пути `/.well-known/acme-challenge/`, который используется Certbot для подтверждения владения доменом;
- HTTPS (порт 443): проксирует запросы к фронтенду (`/`), REST API (`/api/`) и WebSocket (`/ws/`).

Для корректной проксировки WebSocket необходимы специальные директивы: `proxy_http_version 1.1`, `proxy_set_header Upgrade` и

proxy_set_header Connection «upgrade». Без них NGINX разрывает WebSocket-соединения при установке.

2.5. Выпуск SSL-сертификата

Сертификат выпускается через HTTP-01 challenge: Certbot создаёт временный файл в директории /var/www/certbot, NGINX отдаёт его по пути /.well-known/acme-challenge/, серверы Let's Encrypt проверяют файл и выдают сертификат.

Контейнер certbot настроен на автоматическое обновление: он запускает certbot renew каждые 12 часов. Сертификаты монтируются в контейнер NGINX через общий том certbot/conf.

ГЛАВА 3. ДОКУМЕНТАЦИЯ ПО ЗАПУСКУ

Полная документация по запуску представлена в файле README.md в корне репозитория по [адресу](#). Ниже приведён краткий порядок действий.

Шаг 1. Установить Docker и плагин Compose

```
apt update
apt install -y docker-ce docker-ce-cli containerd.io
apt install -y docker-compose-plugin
```

Шаг 2. Клонировать репозиторий

```
git clone https://github.com/A1adriel/chat-app.git
cd chat-app
```

Шаг 3. Запустить контейнеры

Поскольку путь к проекту содержит пробелы, при запуске необходимо указывать имя проекта явно через флаг -p:

```
docker compose -p chatapp up -d --build
```

Шаг 4. Выпустить SSL-сертификат

Сертификат выпускается через системный Certbot методом standalone. Для этого необходимо временно остановить контейнер nginx, чтобы освободить порт 80:

```
docker compose -p chatapp stop nginx
certbot certonly --standalone -d sambuev-17.ru \
--email sambuev.2002@mail.ru --agree-tos \
--no-eff-email --non-interactive
docker compose -p chatapp start nginx
```

Шаг 5. Подключить сертификат к NGINX

Обновить конфигурацию nginx/default.conf, указав пути к сертификатам, и добавить монтирование /etc/letsencrypt в docker-compose.yml. Пересоздать контейнер nginx:

```
docker compose -p chatapp stop nginx  
docker compose -p chatapp rm -f nginx  
docker compose -p chatapp up -d nginx
```

После выполнения всех шагов приложение доступно по адресу <https://sambuev-17.ru>. Обновление сертификата происходит автоматически через системный таймер Certbot.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было разработано и контейнеризовано многокомпонентное веб-приложение AnonChat — анонимный чат в реальном времени. Были решены следующие задачи:

- разработан React-фронтенд с поддержкой WebSocket и двумя экранами (вход и чат);
- реализован FastAPI-бэкенд с REST API, WebSocket-сервером и хранением данных в PostgreSQL;
- написаны Dockerfile для каждого сервиса, включая многоступенчатую сборку фронтенда;
- составлен docker-compose.yml, объединяющий пять сервисов в изолированной сети;
- настроен NGINX как обратный прокси с поддержкой проксирования WebSocket и SSL-терминацией;
- автоматизирован выпуск и обновление SSL-сертификатов Let's Encrypt через Certbot;
- составлена документация по развёртыванию проекта.

Особую сложность представляла настройка проксирования WebSocket-соединений через NGINX, которая требует явной передачи заголовков протокола обновления соединения. Данный аспект наглядно демонстрирует, что контейнеризация сетевых приложений с постоянными соединениями требует более тщательной настройки, чем стандартные HTTP-приложения.

Применение Docker Compose позволило свести процесс развёртывания всего стека к одной команде, а использование Let's Encrypt полностью исключило затраты на SSL-сертификаты при сохранении полного доверия браузеров.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Docker Documentation [Электронный ресурс]. — Режим доступа: <https://docs.docker.com> (дата обращения: 01.06.2025). — Загл. с экрана.
2. FastAPI Documentation [Электронный ресурс]. — Режим доступа: <https://fastapi.tiangolo.com> (дата обращения: 01.06.2025). — Загл. с экрана.
3. React Documentation [Электронный ресурс]. — Режим доступа: <https://react.dev> (дата обращения: 01.06.2025). — Загл. с экрана.
4. NGINX Documentation [Электронный ресурс]. — Режим доступа: <https://nginx.org/en/docs/> (дата обращения: 01.06.2025). — Загл. с экрана.
5. Let's Encrypt Documentation [Электронный ресурс]. — Режим доступа: <https://letsencrypt.org/docs/> (дата обращения: 01.06.2025). — Загл. с экрана.
6. Certbot Documentation [Электронный ресурс]. — Режим доступа: <https://certbot.eff.org/docs/> (дата обращения: 01.06.2025). — Загл. с экрана.
7. PostgreSQL Documentation [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/docs/> (дата обращения: 01.06.2025). — Загл. с экрана.
8. SQLAlchemy Documentation [Электронный ресурс]. — Режим доступа: <https://docs.sqlalchemy.org/> (дата обращения: 01.06.2025). — Загл. с экрана.